Experiment 12 — Graph analytics and sparse linear algebra

Mathilda — execution-speed experiments

Contents

Experiment 12 — Graph analytics and sparse linear algebra

Date: 2026-07-31 · Code: `src/ndarray.c` (`build_axis_selector`), `src/part.c`, `src/list/setops.c`, `src/pack.c` · Result: PageRank 14.1 s → 486 ms; the gather underneath it 389 ms → 15.7 ms

Common method in `README.md`.

Hypothesis

Every sparse and graph kernel in existence is built out of one operation: gather — `y = x[[idx]]`, reading a value array through an index array. A sparse matrix–vector product is a gather, a multiply and a segmented sum. A PageRank iteration is twenty of those. A breadth-first search is a gather of adjacency rows followed by a set difference.

The previous four sweeps never ran one. They measured streaming access — `Total`, `Accumulate`, element-wise arithmetic, stencils — where the address of element i+1 is known before element i is read. A gather is the opposite: the access pattern is data.

Mathilda has no `SparseArray`, so the representation used here is the one a system without it actually uses, and the one every uniform-degree graph produces anyway: a dense `n × d` matrix of neighbour ids. That is exactly CSR with the row pointers implied, and it isolates the gather rather than hiding it inside a sparse-matrix type.

The kernels

100000 nodes × degree 16 = 1.6 × 10^6 edges — a mid-sized citation or road network. The timed graph is random; every value check runs a separate deterministic 4096-node graph, because the three systems cannot be made to draw the same random one.

id	kernel	what it stresses
`gather`	`x[[idx]]`, 1.6 × 10⁶ indices	the primitive, alone
`spmv`	one CSR matrix–vector product	gather + multiply + segmented sum
`pagerank`	20 power iterations	the above, in a loop, with a reduction
`bfs`	5 levels of frontier expansion	row gather + `Union` + `Complement`

What the first run found

	Mathilda	Mathematica	NumPy
Gather, 1.6e6 indices	389 ms	9.7 ms	5.7 ms
SpMV	438 ms	17.9 ms	9.7 ms
PageRank, 20 iterations	14.10 s	355 ms	168 ms
BFS, 5 levels	482 ms	51.9 ms	78.2 ms

40× behind Mathematica and 84× behind NumPy on PageRank — by a wide margin the worst row any of the five sweeps has produced, and the only one where Mathilda was worse than both systems on every kernel in the group.

The cause: nobody packed the index

`ndarray_part` has had a complete fancy-gather path all along — `build_axis_selector` turns a subscript into a list of source positions and a mixed-radix walk copies through it. It accepts an `All`, a `Span`, an `Integer`, and a **`List`** of integer positions.

It does not accept a packed list of integer positions.

That distinction was invisible when the fast path was written and is fatal now, because every producer of an index array returns a packed one: `Range`, `Flatten`, `RandomInteger`, `Ordering`-style constructions, and any arithmetic on them. So the spec arrived as an `EXPR_NDARRAY`, fell through to `NDPART_DEGRADE`, and the whole `Part` was delisted — materialising both the 1.6 × 10⁶-element index array and the 100000-element source, gathering through boxed expressions, and repacking.

One arm in `build_axis_selector`, twenty lines, reading the int64 buffer directly:

	before	after
`x[[idx]]`, 1.6e6 indices into 100000	389 ms	15.7 ms
`m[[rows]]`, 50000 rows of a 100000×16	506 ms	13.7 ms

The gate is deliberately narrow — rank 1, `int64`, and `is_packed_list` rather than `is_ndarray`, so a visible `NDArray[…]` behaves exactly as it did (the List path would not have accepted one either). A `Real` position, an out-of-range one, an empty list and a rank-2 index array all degrade and produce the ordinary answer, diagnostics included.

The set operations were the other half

BFS is literally

```
next = Complement[Union[Flatten[adj[[frontier]]]], visited]
```

once per level. `Union`, `Intersection` and `Complement` were written generically: `expr_copy` every element, `qsort` through a function pointer calling `expr_compare`, then dedup with `expr_eq`. On integer labels — which is what graph vertices are — that is one allocation and two indirect calls per element.

A sorted int64 merge replaces it:

	before	after
`Union` of 10^6 integers	745 ms	40.9 ms
`Complement` of 10^6 against 10^5	773 ms	42.7 ms

Restricted to integers on purpose. `0.` and `-0.` compare equal and print differently, so which of two equal reals survives a dedup is a question the buffer path and the List path must not be allowed to answer differently. A set operation whose result depends on that is not a fast path, it is a second implementation. Integers have no such pair.

Results

Benchmark	before	after	Mathematica 14.0	NumPy 2.4.4	
Gather, 1.6e6 indices into 100000	389 ms	15.7 ms	9.7 ms	5.7 ms	24.8×
SpMV, 100000 × 16 CSR	438 ms	20.5 ms	17.9 ms	9.7 ms	21.4×
PageRank, 1.6e6 edges, 20 iterations	14.10 s	486 ms	355 ms	168 ms	29.0×
BFS, 5 levels	482 ms	121 ms	51.9 ms	78.2 ms	4.0×

PageRank went from 40× behind Mathematica to 1.37× behind it, and from 84× behind NumPy to 2.9×. All four values agree across all three systems.

What is still open

- The remaining 2–3× against NumPy is the gather itself, and it is a memory problem rather than a dispatch one: `build_axis_selector` materialises a `int64*` position array of the full index length before the copy loop runs, so a 1.6 × 10^6-index gather writes 13 MB of positions it then reads once. Reading the index buffer in the copy loop removes that pass entirely. This is the same allocation the fourth sweep's item 9.5 identified for spans.
- BFS is the weakest row at 2.33× Mathematica, and the cause is that the frontier gather `adj[[frontier]]` produces a rank-2 result which is then `Flattened` — two passes over 2.9 × 10^6 elements where one would do.
- No **`SparseArray`**. Uniform degree is a real and common case, but a general sparse matrix needs a row-pointer array and a segmented reduction that does not go through `Partition`. That is a data structure, not a fast path.

Why Mathilda is not the fastest here, and what it would take

row	Mathilda	best other	gap	cause
Gather, 1.6e6 indices	15.7 ms	5.7 ms (NumPy)	2.73×	a full-length position array is written before the copy loop
SpMV	20.5 ms	9.7 ms (NumPy)	2.12×	the gather, plus two unfused passes
PageRank, 20 iterations	486 ms	168 ms (NumPy)	2.89×	twenty of the above
BFS, 5 levels	121 ms	51.9 ms (WL)	2.33×	the row gather is materialised into a rank-2 result and then

All four are now within 3× of the best of the other two, from 40× and 84×. The remainder is three specific things, in value order.

The road to fastest

1. Do not materialise the positions. `build_axis_selector` allocates an `int64*` of the full index length and fills it, and only then does the copy loop read it. A 1.6 × 10⁶-index gather therefore writes 13 MB it reads exactly once — a whole extra pass over more memory than the answer. Reading the index buffer inside the copy loop removes it. Expected: most of the 2.7×, since the gather is otherwise a pure random-access read that NumPy does no differently. This is the same allocation as plan item 9.5's spans, so one change closes both.
2. Thread the gather. The copy loop is serial. A gather is embarrassingly parallel and memory-latency-bound, which is precisely the case where threading helps most — `nd_parallel_for` already exists and every other elementwise path uses it.
3. Fuse the SpMV triple. `gw * gv[[gflat]]` then `Partition` then `Total[…, {2}]` is three passes over 1.6 × 10⁶ elements plus a rank-2 intermediate that exists only to be summed. A segmented-reduction kernel that takes a value array, an index array and a row width would do it in one, and is a small, self-contained addition to `ndreduce.c`. Expected: SpMV and PageRank at or below NumPy, since NumPy makes the same three passes and has no fused kernel for this either.
4. Then the BFS row. `adj[[frontier]]` builds a rank-2 result which is immediately `Flattened` — two passes where a flattening gather is one. A `Flatten[a[[rows]]]` recognition, or simply letting the rank-2 gather write a rank-1 buffer when its consumer is `Flatten`, closes it.

With 1–3 the graph rows should lead: NumPy's advantage here is not a better gather, it is a gather that does not write a position array first.

`SparseArray` is deliberately not on this list. Uniform degree is a real and common case and it is what this experiment measures; a general sparse matrix wants a row-pointer array and the same segmented reduction as item 3, so item 3 is the prerequisite either way.

Verification

- `test_fifth_sweep_fast_paths` in `tests/test_packed_list.c`: ~50 differential cases over the gather and the set operations, each the same source evaluated with automatic packing on and off, so the buffer path is compared against the interpreter's List path on identical input. Every form each path must decline is included — `Real` positions, out-of-range, empty, rank-2 indices, `Real` set operands, a `SameTest`, and mixed packed/plain operands.
- `make check-packed-aware` passes; the three set-operation heads are registered in both the aware and the `INT64_OK` lists.
- Every benchmark value agrees across Mathilda, Mathematica and NumPy.